UNIVERSIDAD NACIONAL DEL ALTIPLANO – PUNO
Facultad de Ingeniería Mecánica Eléctrica, Electrónica y Sistemas
Escuela Profesional de Ingeniería de Sistemas

El problema de la mochila: algoritmos voraces vs. programación dinámica

Tarea 7_2 – Algoritmos Voraces: Problemas

Estudiante: Emerson Aldair Flores Flores
Código de matrícula: 240439
Semestre: VI
Fecha de entrega: 12 de octubre de 2026

Puno, Perú – 2026

Índice

1 Introducción **2**

2 Marco teórico **2**
2.1 Algoritmos voraces . . . 2
2.2 Programación dinámica . . . 2
2.3 Problema de los billetes (antecedente) . . . 2

3 Formulación del problema **3**

4 Solución con algoritmos voraces **3**
4.1 Mochila fraccionada . . . 3
4.2 Mochila 0/1: el voraz deja de ser óptimo . . . 3

5 Solución con programación dinámica (mochila 0/1) **4**
5.1 Subproblema y recurrencia . . . 4
5.2 Reconstrucción de la solución . . . 4
5.3 Ejemplo con tabla . . . 4
5.4 Aplicación al ejemplo del libro . . . 5
5.5 Complejidad . . . 5

6 Comparación **5**

7 Resultados experimentales **5**
7.1 Frecuencia de error del voraz 0/1 . . . 5
7.2 Tiempos de ejecución . . . 6

8 Conclusiones **6**

A Código en Python **7**

B Código en C++ **9**

1 Introducción

El capítulo 21 de *Algoritmos a fondo* (Sznajdleder) presenta tres estrategias algorítmicas: divide y conquista, algoritmos voraces (*greedy*) y programación dinámica. Sobre el problema de la mochila, el texto resuelve únicamente la variante **fraccionada** con un algoritmo voraz, y sobre el problema de los billetes muestra que el enfoque voraz puede fallar y que la programación dinámica lo corrige.

Este informe une ambas ideas: se plantea el problema de la mochila en sus dos variantes (fraccionada y 0/1), se resuelve con un algoritmo voraz y con programación dinámica, y se comparan ambos enfoques en corrección, complejidad y uso de memoria. Las implementaciones se realizaron en **C++** y **Python**.

Objetivo general. Comparar el desempeño y la corrección de los algoritmos voraces y de la programación dinámica al resolver el problema de la mochila.

Objetivos específicos.

- Resolver el ejemplo del libro (capacidad de 40 kg) con ambos enfoques.
- Determinar en qué variante el algoritmo voraz es óptimo y en cuál no.
- Medir experimentalmente la frecuencia de error del voraz y los tiempos de ejecución.

2 Marco teórico

2.1 Algoritmos voraces

Un algoritmo voraz construye la solución evaluando uno a uno los elementos de un *conjunto de candidatos*; cada elemento se toma o se descarta de forma definitiva, sin reconsiderar la decisión. Se identifican cuatro funciones:

- **Selección**: elige el candidato más apropiado.
- **Factibilidad**: verifica si el candidato puede incluirse en la solución.
- **Solución**: verifica si el conjunto construido resuelve el problema.
- **Objetivo**: la magnitud que se desea optimizar.

2.2 Programación dinámica

Se basa en el **principio de optimalidad** de Bellman: *en una secuencia óptima de decisiones, toda subsecuencia también es óptima.* El problema se divide en subproblemas **no independientes**; el resultado de cada uno se almacena en una tabla para no recalcularlo. Implica menos operaciones a cambio de mayor uso de memoria.

2.3 Problema de los billetes (antecedente)

Con denominaciones \$10, \$40 y \$60 y un monto de \$80, el voraz entrega 1 billete de \$60 y 2 de \$10 (3 billetes), mientras que el óptimo son 2 billetes de \$40. Esto muestra que *la elección localmente mejor no siempre conduce al óptimo global*, situación que reaparece en la mochila 0/1.

3 Formulación del problema

Se tiene una mochila de capacidad P y n objetos, cada uno con peso p_i y valor v_i. Se desea

$$\max \sum_{i=1}^{n} x_i\, v_i \qquad \text{sujeto a} \qquad \sum_{i=1}^{n} x_i\, p_i \le P.$$

- **Mochila fraccionada:** $x_i \in [0, 1]$ (los objetos pueden dividirse). Es la variante del libro.
- **Mochila 0/1:** $x_i \in \{0, 1\}$ (cada objeto se toma completo o no se toma).

Datos del ejemplo del libro. $P = 40$ kg.

Elemento	Valor ($)	Peso (kg)	$/kg
A	50	36	1,39
B	48	30	1,60
C	30	20	1,50

Tabla 1: Elementos del ejemplo y su relación valor/peso.

Nota: la relación de B es 48/30 = 1,60 (el texto original consigna 1,66, pero el orden B > C > A no cambia).

4 Solución con algoritmos voraces

4.1 Mochila fraccionada

Selección: el objeto con mayor relación v_i/p_i. **Factibilidad:** se carga la fracción $x_i = \min(1, \text{disponible}/p_i)$.

1. Ordenar por v/p decreciente: B, C, A.
2. Cargar B completo (30 kg). Quedan 10 kg.
3. C pesa 20 kg: se carga $\frac{1}{2}$ de C (10 kg). Mochila llena.
4. A no se carga.

Valor total: $1 \cdot 48 + \frac{1}{2} \cdot 30 = \mathbf{63}$ $.

Por qué es óptimo. Por un argumento de intercambio: si una solución óptima dejara espacio sin usar de un objeto de mayor relación v/p y tuviera cargado parte de uno de menor relación, intercambiar esa parte aumentaría el valor sin violar la capacidad. Por lo tanto la solución voraz ordenada por v/p es óptima en la variante fraccionada.

4.2 Mochila 0/1: el voraz deja de ser óptimo

Cuando no se puede fraccionar, se usan los mismos criterios pero se descarta el objeto si no cabe entero. Se probaron tres criterios de selección con el mismo ejemplo:

Criterio de selección	Selección	Valor ($)	¿Óptimo?
Mayor v/p	{B}	48	No
Mayor valor	{A}	50	Sí (por casualidad)
Menor peso	{C}	30	No

Tabla 2: Voraz 0/1 con tres criterios para $P = 40$.

Con el criterio v/p se elige B (30 kg) y luego ni C (20 kg) ni A (36 kg) caben en los 10 kg restantes: el valor es \$48, aunque existe una solución mejor (solo A, \$50). El voraz tomó una decisión irrevocable que bloqueó una mejor combinación.

Complejidad. El ordenamiento domina: $O(n \log n)$ en tiempo y $O(1)$ de memoria adicional (además del arreglo ordenado).

5 Solución con programación dinámica (mochila 0/1)

5.1 Subproblema y recurrencia

Sea $m[i][w]$ el mejor valor que se puede obtener usando solo los primeros i objetos con una capacidad w:

$$m[i][w] = \begin{cases} 0 & \text{si } i = 0 \text{ o } w = 0, \\ m[i-1][w] & \text{si } p_i > w, \\ \max\big(m[i-1][w],\ m[i-1][w-p_i] + v_i\big) & \text{si } p_i \le w. \end{cases}$$

La respuesta es $m[n][P]$. Es el mismo estilo de tabla que el libro usa para los billetes: cada celda reutiliza celdas de la fila anterior, y las dos opciones del max corresponden a *no usar* o *usar* el objeto i.

5.2 Reconstrucción de la solución

La tabla solo da el valor óptimo. Para saber qué objetos se cargaron se recorre desde (n, P): si $m[i][w] \neq m[i-1][w]$ el objeto i fue usado y se hace $w \leftarrow w - p_i$; en caso contrario no se usó. Esto equivale a la matriz booleana `utiliza` del libro.

5.3 Ejemplo con tabla

Para mostrar la tabla completa se usa un caso pequeño con $P = 10$: A(5 kg, \$10), B(4 kg, \$40), C(6 kg, \$30), D(3 kg, \$50).

	0	1	2	3	4	5	6	7	8	9	10
∅	0	0	0	0	0	0	0	0	0	0	0
A	0	0	0	0	0	10	10	10	10	10	10
B	0	0	0	0	40	40	40	40	40	50	50
C	0	0	0	0	40	40	40	40	40	50	70
D	0	0	0	50	50	50	50	**90**	90	90	**90**

Tabla 3: Tabla $m[i][w]$ para $P = 10$. El valor óptimo es $m[4][10] = 90$, con los objetos B y D (7 kg).

5.4 Aplicación al ejemplo del libro

Con $P = 40$ y los objetos A, B, C la programación dinámica evalúa las combinaciones factibles: A (36 kg, \$50), B (30 kg, \$48), C (20 kg, \$30); B+C pesa 50 kg y no cabe. El máximo es $m[3][40] = \mathbf{50}$ con {A}, superior a los \$48 del voraz.

5.5 Complejidad

- **Tiempo:** $O(nP)$. Es *pseudopolinomial*: depende del valor numérico de P y no solo de la cantidad de datos.
- **Memoria:** $O(nP)$ con reconstrucción. Si solo se necesita el valor, basta un arreglo unidimensional de tamaño $P + 1$ recorrido de derecha a izquierda: $O(P)$.

6 Comparación

	Voraz (fraccionada)	**Voraz (0/1)**	**Prog. dinámica (0/1)**
Resultado en el ejemplo ($P = 40$)	\$63 (óptimo)	\$48 con v/p (no óptimo)	\$50 (óptimo)
¿Garantiza el óptimo?	Sí	No	Sí
Decisiones	Irrevocables	Irrevocables	Se evalúan todas las opciones por celda
Tiempo	$O(n \log n)$	$O(n \log n)$	$O(nP)$
Memoria	$O(1)$ extra	$O(1)$ extra	$O(nP)$ (u $O(P)$ sin reconstrucción)
Subproblemas	No se almacenan	No se almacenan	Se almacenan en tabla
Objetos fraccionables	Sí	No	No
Facilidad de implementación	Alta	Alta	Media

Tabla 4: Comparación entre los enfoques.

7 Resultados experimentales

Se implementaron los tres algoritmos en C++ y en Python (código en los anexos).

7.1 Frecuencia de error del voraz 0/1

Se generaron 2000 instancias aleatorias ($3 \le n \le 8$, pesos en $[1, 20]$, valores en $[1, 60]$, $10 \le P \le 50$, semilla 2026) y se comparó el voraz por v/p con la programación dinámica.

	Python	C++
Instancias en que el voraz no fue óptimo	379 (18,9 %)	368 (18,4 %)
Pérdida relativa media en esos casos	10,4 %	–

Tabla 5: Fallos del voraz 0/1. Las diferencias entre lenguajes se deben a los distintos generadores aleatorios.

Aproximadamente una de cada cinco instancias da un resultado subóptimo con el voraz; la programación dinámica fue siempre óptima.

7.2 Tiempos de ejecución

		Python (ms)		C++ (ms)	
n	P	Voraz	Dinámica	Voraz	Dinámica
100	1000	0,030	14,985	0,02	0,35
200	2000	0,057	63,768	0,03	1,39
400	4000	0,107	285,464	0,07	5,41

Tabla 6: Tiempos medidos (una ejecución por caso; los valores varían según el equipo).

Al duplicar n y P simultáneamente, el tiempo de la programación dinámica se multiplica por aproximadamente 4 (coherente con $O(nP)$), mientras que el voraz crece casi linealmente. Python es notablemente más lento que C++ en la programación dinámica por el costo del intérprete en los bucles anidados.

8 Conclusiones

1. En la **mochila fraccionada** el algoritmo voraz ordenado por v/p es óptimo, simple y muy eficiente ($O(n \log n)$). En el ejemplo del libro obtiene \$63.

2. En la **mochila 0/1** el voraz es solo una heurística: con $P = 40$ entrega \$48 cuando el óptimo es \$50, y en las pruebas aleatorias falló en cerca del 19 % de las instancias.

3. La **programación dinámica** garantiza el óptimo en la variante 0/1 gracias al principio de optimalidad y al almacenamiento de subproblemas, pero con costo $O(nP)$ en tiempo y memoria, que crece con la capacidad.

4. Esto confirma lo señalado en el resumen del capítulo: la programación dinámica resuelve problemas que los métodos voraces, en ocasiones, no pueden resolver. La elección depende de la variante del problema: voraz si los objetos son fraccionables o si basta una solución aproximada rápida; programación dinámica si se exige el óptimo exacto y P es manejable.

Referencias

[1] Sznajdleder, P. A. *Algoritmos a fondo*. Alfaomega. Capítulo 21: Estrategia algorítmica.

[2] Cormen, T., Leiserson, C., Rivest, R., Stein, C. *Introduction to Algorithms*. MIT Press (capítulos sobre algoritmos voraces y programación dinámica).

A Código en Python

```python
"""
Problema de la mochila: algoritmos voraces vs programacion dinamica.
Tarea 7_2 - Algoritmos Voraces - Problemas
Autor: Emerson Aldair Flores Flores (cod. 240439) - UNAP
"""
import random
import time
from dataclasses import dataclass


@dataclass
class Elemento:
    nombre: str
    peso: int
    valor: int
    cantidad: float = 0.0  # fraccion cargada (0..1)

    @property
    def relacion(self) -> float:
        return self.valor / self.peso


# ------------------------------------------------------------
# 1) VORAZ - mochila FRACCIONADA (optimo)  O(n log n)
# ------------------------------------------------------------
def voraz_fraccionada(capacidad, elementos):
    items = sorted(elementos, key=lambda e: e.relacion, reverse=True)
    disponible = capacidad
    total = 0.0
    sel = []
    for e in items:
        if disponible <= 0:
            break
        x = min(1.0, disponible / e.peso)
        sel.append((e.nombre, x))
        total += x * e.valor
        disponible -= x * e.peso
    return total, sel


# ------------------------------------------------------------
# 2) VORAZ - mochila 0/1 (heuristica, NO siempre optima)  O(n log n)
#    criterio: 'relacion' (valor/peso), 'valor' o 'peso' (menor primero)
# ------------------------------------------------------------
def voraz_01(capacidad, elementos, criterio="relacion"):
    if criterio == "relacion":
```

```
        clave = lambda e: -e.relacion
    elif criterio == "valor":
        clave = lambda e: -e.valor
    else:
        clave = lambda e: e.peso
    disponible = capacidad
    total = 0
    sel = []
    for e in sorted(elementos, key=clave):
        if e.peso <= disponible:  # funcion de factibilidad
            sel.append(e.nombre)
            total += e.valor
            disponible -= e.peso
    return total, sel


# ----------------------------------------------------------------
# 3) PROGRAMACION DINAMICA - mochila 0/1 (optimo)  O(n*P) tiempo y espacio
#    m[i][w] = mejor valor usando los primeros i elementos con capacidad w
#    m[i][w] = max( m[i-1][w], m[i-1][w-p_i] + v_i )   si p_i <= w
# ----------------------------------------------------------------
def dinamica_01(capacidad, elementos):
    n = len(elementos)
    m = [[0] * (capacidad + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        p, v = elementos[i - 1].peso, elementos[i - 1].valor
        for w in range(capacidad + 1):
            m[i][w] = m[i - 1][w]
            if p <= w and m[i - 1][w - p] + v > m[i][w]:
                m[i][w] = m[i - 1][w - p] + v
    # reconstruccion de la solucion
    sel, w = [], capacidad
    for i in range(n, 0, -1):
        if m[i][w] != m[i - 1][w]:
            sel.append(elementos[i - 1].nombre)
            w -= elementos[i - 1].peso
    sel.reverse()
    return m[n][capacidad], sel, m


def imprimir_tabla(m, capacidad, elementos):
    print("      " + " ".join(f"{w:3d}" for w in range(capacidad + 1)))
    for i, fila in enumerate(m):
        etiqueta = "  -  " if i == 0 else f"  {elementos[i-1].nombre}  "
        print(etiqueta + " ".join(f"{x:3d}" for x in fila))


def main():
    # ---- Ejemplo del libro (capitulo 21) ----
    capacidad = 40
    elementos = [Elemento("A", 36, 50), Elemento("B", 30, 48), Elemento("C", 20, 30)]

    print("=== Ejemplo del libro: capacidad =", capacidad, "kg ===")
    v, s = voraz_fraccionada(capacidad, elementos)
    print(f"Voraz fraccionada     : valor = {v:.2f}  seleccion = {s}")
    for crit in ("relacion", "valor", "peso"):
        v, s = voraz_01(capacidad, elementos, crit)
        print(f"Voraz 0/1 ({crit:8s}) : valor = {v}  seleccion = {s}")
```

```python
    v, s, m = dinamica_01(capacidad, elementos)
    print(f"Prog. dinamica 0/1    : valor = {v}  seleccion = {s}")

    # ---- Ejemplo pequeno para mostrar la tabla DP ----
    print("\n=== Tabla DP (capacidad 10) ===")
    pequeno = [Elemento("A", 5, 10), Elemento("B", 4, 40), Elemento("C", 6, 30),
    Elemento("D", 3, 50)]
    v, s, m = dinamica_01(10, pequeno)
    imprimir_tabla(m, 10, pequeno)
    print("valor optimo =", v, "seleccion =", s)

    # ---- Experimento: frecuencia de fallo del voraz 0/1 ----
    random.seed(2026)
    pruebas, fallos, brecha = 2000, 0, 0.0
    for _ in range(pruebas):
        n = random.randint(3, 8)
        its = [Elemento(chr(65 + k), random.randint(1, 20), random.randint(1, 60)) for
     k in range(n)]
        cap = random.randint(10, 50)
        vg, _ = voraz_01(cap, its, "relacion")
        vd, _, _ = dinamica_01(cap, its)
        if vg < vd:
            fallos += 1
            brecha += (vd - vg) / vd
    print(f"\n=== Experimento ({pruebas} instancias aleatorias) ===")
    print(f"El voraz 0/1 por relacion v/p fallo en {fallos} ({100*fallos/pruebas:.1f
    }%)")
    print(f"Perdida relativa media en los fallos: {100*brecha/max(fallos,1):.1f}%")

    # ---- Experimento: tiempos ----
    print("\n=== Tiempos (ms) ===")
    print(f"{'n':>6} {'P':>7} {'voraz':>10} {'dinamica':>12}")
    for n, P in [(100, 1000), (200, 2000), (400, 4000)]:
        its = [Elemento(str(k), random.randint(1, 50), random.randint(1, 500)) for k
    in range(n)]
        t0 = time.perf_counter(); voraz_01(P, its); t1 = time.perf_counter()
        dinamica_01(P, its); t2 = time.perf_counter()
        print(f"{n:6d} {P:7d} {1000*(t1-t0):10.3f} {1000*(t2-t1):12.3f}")


if __name__ == "__main__":
    main()
```

B Código en C++

```cpp
// Problema de la mochila: algoritmos voraces vs programacion dinamica.
// Tarea 7_2 - Algoritmos Voraces - Problemas
// Autor: Emerson Aldair Flores Flores (cod. 240439) - UNAP
// Compilar: g++ -std=c++17 -O2 mochila.cpp -o mochila
#include <algorithm>
#include <chrono>
#include <cstdlib>
#include <iomanip>
#include <iostream>
#include <string>
#include <vector>
```

```cpp
using namespace std;

struct Elemento {
    string nombre;
    int peso;
    int valor;
    double relacion() const { return (double)valor / peso; }
};

// 1) VORAZ - mochila fraccionada (optimo) O(n log n)
double vorazFraccionada(int capacidad, vector<Elemento> v, vector<pair<string,double
    >>& sel) {
    sort(v.begin(), v.end(), [](const Elemento& a, const Elemento& b) {
        return a.relacion() > b.relacion();
    });
    double disponible = capacidad, total = 0;
    for (auto& e : v) {
        if (disponible <= 0) break;
        double x = min(1.0, disponible / e.peso);
        sel.push_back({e.nombre, x});
        total += x * e.valor;
        disponible -= x * e.peso;
    }
    return total;
}

// 2) VORAZ - mochila 0/1 (heuristica) O(n log n)
// criterio: 0 = valor/peso, 1 = valor, 2 = peso (menor primero)
int vorazCeroUno(int capacidad, vector<Elemento> v, int criterio, vector<string>& sel)
     {
    sort(v.begin(), v.end(), [criterio](const Elemento& a, const Elemento& b) {
        if (criterio == 0) return a.relacion() > b.relacion();
        if (criterio == 1) return a.valor > b.valor;
        return a.peso < b.peso;
    });
    int disponible = capacidad, total = 0;
    for (auto& e : v) {
        if (e.peso <= disponible) {          // funcion de factibilidad
            sel.push_back(e.nombre);
            total += e.valor;
            disponible -= e.peso;
        }
    }
    return total;
}

// 3) PROGRAMACION DINAMICA - mochila 0/1 (optimo) O(n*P)
// m[i][w] = max(m[i-1][w], m[i-1][w-p_i] + v_i)
int dinamicaCeroUno(int capacidad, const vector<Elemento>& v, vector<string>& sel,
                    vector<vector<int>>* tabla = nullptr) {
    int n = v.size();
    vector<vector<int>> m(n + 1, vector<int>(capacidad + 1, 0));
    for (int i = 1; i <= n; i++) {
        int p = v[i - 1].peso, val = v[i - 1].valor;
        for (int w = 0; w <= capacidad; w++) {
            m[i][w] = m[i - 1][w];
            if (p <= w) m[i][w] = max(m[i][w], m[i - 1][w - p] + val);
        }
```

```
    }
    int w = capacidad;                          // reconstruccion
    for (int i = n; i >= 1; i--) {
        if (m[i][w] != m[i - 1][w]) {
            sel.push_back(v[i - 1].nombre);
            w -= v[i - 1].peso;
        }
    }
    reverse(sel.begin(), sel.end());
    int res = m[n][capacidad];
    if (tabla) *tabla = m;
    return res;
}

void mostrar(const vector<string>& s) {
    cout << "{";
    for (size_t i = 0; i < s.size(); i++) cout << (i ? ", " : "") << s[i];
    cout << "}";
}

int main() {
    int capacidad = 40;
    vector<Elemento> el = {{"A", 36, 50}, {"B", 30, 48}, {"C", 20, 30}};

    cout << "=== Ejemplo del libro: capacidad = " << capacidad << " kg ===\n";
    vector<pair<string,double>> sf;
    double vf = vorazFraccionada(capacidad, el, sf);
    cout << fixed << setprecision(2) << "Voraz fraccionada     : valor = " << vf << "
     ";
    for (auto& p : sf) cout << p.first << "=" << p.second << " ";
    cout << "\n";

    const char* nombres[] = {"relacion", "valor   ", "peso    "};
    for (int c = 0; c < 3; c++) {
        vector<string> s;
        int v = vorazCeroUno(capacidad, el, c, s);
        cout << "Voraz 0/1 (" << nombres[c] << ") : valor = " << v << "  seleccion = "
    ;
        mostrar(s); cout << "\n";
    }
    vector<string> sd;
    int vd = dinamicaCeroUno(capacidad, el, sd);
    cout << "Prog. dinamica 0/1    : valor = " << vd << "  seleccion = ";
    mostrar(sd); cout << "\n";

    // Experimento: frecuencia de fallo del voraz 0/1
    srand(2026);
    int pruebas = 2000, fallos = 0;
    for (int t = 0; t < pruebas; t++) {
        int n = 3 + rand() % 6;
        vector<Elemento> its;
        for (int k = 0; k < n; k++)
            its.push_back({string(1, 'A' + k), 1 + rand() % 20, 1 + rand() % 60});
        int cap = 10 + rand() % 41;
        vector<string> s1, s2;
        if (vorazCeroUno(cap, its, 0, s1) < dinamicaCeroUno(cap, its, s2)) fallos++;
    }
    cout << "\n=== Experimento (" << pruebas << " instancias) ===\n";
```

```
124     cout << "El voraz 0/1 fallo en " << fallos << " (" << 100.0 * fallos / pruebas <<
        "%)\n";
125 
126     // Tiempos
127     cout << "\n=== Tiempos (ms) ===\n";
128     cout << setw(6) << "n" << setw(8) << "P" << setw(12) << "voraz" << setw(12) << "
        dinamica\n";
129     int casos[3][2] = {{100, 1000}, {200, 2000}, {400, 4000}};
130     for (auto& c : casos) {
131         vector<Elemento> its;
132         for (int k = 0; k < c[0]; k++) its.push_back({to_string(k), 1 + rand() % 50, 1
         + rand() % 500});
133         vector<string> s;
134         auto t0 = chrono::high_resolution_clock::now();
135         vorazCeroUno(c[1], its, 0, s);
136         auto t1 = chrono::high_resolution_clock::now();
137         s.clear();
138         dinamicaCeroUno(c[1], its, s);
139         auto t2 = chrono::high_resolution_clock::now();
140         cout << setw(6) << c[0] << setw(8) << c[1] << setw(12)
141              << chrono::duration<double, milli>(t1 - t0).count() << setw(12)
142              << chrono::duration<double, milli>(t2 - t1).count() << "\n";
143     }
144     return 0;
145 }
```